

Actividad Evaluable 2

Descripción

PROGRAMA	Ingeniería de sistema
ASIGNATURA	IS0287 - PROGRAMACIÓN INTERMEDIA
Carácter	Individual
Nombre del Estudiante	
Código estudiante	

Enunciado:

Actividad Práctica: Implementación de Miembros de una Clase en Java

Objetivo: El estudiante aprenderá a configurar un entorno de desarrollo Java en la nube (GitHub Codespaces) y aplicará los conceptos fundamentales de la Programación Orientada a Objetos (POO) mediante la creación de una clase que integre atributos, métodos y constructores.

1. Configuración del Entorno en la Nube: GitHub Codespaces

GitHub Codespaces es un entorno de desarrollo instantáneo y completo alojado en la nube. Proporciona una máquina virtual preconfigurada con las herramientas necesarias para programar en Java, como el JDK y el editor Visual Studio Code, sin necesidad de instalaciones locales.

Para crear un proyecto Java en Codespaces, se deben seguir estos pasos:

1. **Crear un Repositorio:** En GitHub.com, se crea un nuevo repositorio. Se recomienda inicializarlo con un archivo README.md.
2. **Abrir en Codespaces:** Desde la página principal del repositorio, se presiona la tecla "." (punto) o se hace clic en el botón verde "<> Code" y se selecciona la pestaña "Codespaces". Luego, se crea un nuevo codespace.
3. **Configurar el Contenedor de Desarrollo:** Al abrirse el codespace, se accede a la Paleta de Comandos ("Ctrl+Shift+P" o "Cmd+Shift+P"). Se busca y ejecuta el comando "Codespaces: Add Dev Container Configuration Files". Se selecciona "Java" como definición y se elige la versión deseada de Java. Esto genera un archivo ".devcontainer/devcontainer.json" que configura el entorno.
4. **Crear la Estructura del Proyecto:** En el explorador de archivos del codespace, se crea una nueva carpeta para el proyecto (por ejemplo, "src"). Dentro de ella, se crea el archivo ".java" para la clase principal (por ejemplo, "Principal.java" y "Libro.java").

2. Fundamentos Teóricos: Miembros de una Clase

En POO, una clase es una plantilla que define la estructura y el comportamiento de un tipo de objeto. Sus miembros principales son:

- **Atributos (o Campos):** Son variables que definen el estado o las propiedades de un objeto. Cada objeto (instancia de la clase) tiene su propia copia de los atributos no estáticos.
- **Métodos:** Definen el comportamiento o las operaciones que un objeto puede realizar. Son funciones asociadas a la clase.
- **Constructores:** Son métodos especiales llamados automáticamente al crear un nuevo objeto. Su propósito es inicializar sus atributos, asegurando que el objeto comience en un estado válido.

3. Propuesta de Actividad Práctica: Clase "Libro"

Se propone implementar una clase "Libro" que modele las propiedades básicas de un libro en una biblioteca.

Requisitos de Implementación:

- **Atributos:** La clase debe declarar atributos privados para garantizar el encapsulamiento. Ejemplos: titulo (String), autor (String), numeroPaginas (int), isbn (String final) y un atributo estático contadorLibros (int).
- **Constructores:**
 - Un constructor por defecto que inicialice los atributos con valores predeterminados.
 - Un constructor parametrizado que reciba y asigne valores para titulo, autor y numeroPaginas.
 - Un constructor sobrecargado que solo reciba titulo y autor.
- **Métodos:**
 - **Métodos de acceso (Getters):** Como getTitulo(), para leer los atributos privados.
 - **Métodos de modificación (Setters):** Como setNumeroPaginas(int), para cambiar atributos de forma controlada (no aplicable para tipo final).

- **Métodos de comportamiento:** Como mostrarInfo(), que imprima los detalles del libro.
- **Un método estático getTotalLibros()** que devuelva el valor del contador estático.

Esqueleto de Código de Ejemplo:

```
public class Libro {
    // Atributos
    private String titulo;
    private String autor;
    private int numeroPaginas;
    private final String isbn;
    private static int contadorLibros = 0;

    // Constructor por defecto
    public Libro() {
        this.titulo = "Desconocido";
        this.autor = "Desconocido";
        this.numeroPaginas = 0;
        this.isbn = "000-00000000000";
        contadorLibros++;
    }

    // Constructor parametrizado
    public Libro(String titulo, String autor, int numeroPaginas, String isbn) {
        this.titulo = titulo;
        this.autor = autor;
        this.numeroPaginas = numeroPaginas;
        this.isbn = isbn; // Atributo final se inicializa aquí
        contadorLibros++;
    }

    // Métodos getters, setters y de comportamiento
    public String getTitulo() { return titulo; }
    public void setNumeroPaginas(int paginas) { this.numeroPaginas = paginas; }
```

```

public void mostrarInfo() {
    System.out.println("'" + titulo + "' por " + autor + " (" + numeroPaginas + "
páginas). ISBN: " + isbn);
}
public static int getTotalLibros() { return contadorLibros; }
}

```

- **Verificación y Ejecución**

Para probar la implementación, se debe crear una clase Principal con el método "main". En este método, se instancian objetos de la clase "Libro" utilizando los diferentes constructores y se llaman a sus métodos para verificar su funcionamiento.

Ejemplo de Clase de Prueba:

```

public class Principal {
    public static void main(String[] args) {
        Libro libro1 = new Libro(); // Usa constructor por defecto
        Libro libro2 = new Libro("Cien años de soledad", "Gabriel García Márquez",
432, "978-8437604947");

        libro1.mostrarInfo();
        libro2.mostrarInfo();

        libro2.setNumeroPaginas(450); // Modifica un atributo
        System.out.println("El título del libro2 es: " + libro2.getTitulo());

        System.out.println("Total de libros creados: " + Libro.getTotalLibros()); //
Accede al método estático
    }
}

```

Compilación y Ejecución en Codespaces: En la terminal integrada de Codespaces, se navega hasta la carpeta "src" y se ejecuta:

- `javac Principal.java Libro.java`
- `java Principal`

Se debe observar la salida en consola generada por los métodos "mostrarInfo()" y "println", confirmando la correcta creación e interacción de los objetos.

Esta actividad integra la configuración de un entorno profesional en la nube con la aplicación práctica y tangible de los pilares de la POO en Java.

Criterios de Evaluación

Criterio 1: Configuración del Entorno de Desarrollo en la Nube

Criterio 2: Definición de Atributos y Encapsulamiento

Criterio 3: Implementación y Lógica de Constructores

Criterio 4: Historial de Cambios y Versionamiento

Criterio 5: Acceso y Visibilidad del Repositorio